Note on A Probabilistic Instrument for Studying Reasoning Traces and Testing the Ordering and Markovian Assumption Behind Them

Written by Kian Maleki

Problem

When LLM models produce a response to a prompt, the user can turn on the thinking mode. The thinking mode generates a long text that helps the models to generate a more accurate answer (I call this external reasoning against the internal reasoning that happens in a forward pass without generating any thinking text). The long thinking text is not reported back to the user. By using training the model learns that generating the thinking text produces a better answer. This ability of the model empowers the models to answer more complicated questions.

If the thinking text is too long, it becomes human unreadable and unchackable. However, there may be interesting information in the thinking text. Here are two examples: 1- the model solves or fails to solve a math problem but throughout the thinking process it develops a novel technique that only exists in its thinking text. 2- The model concludes that a harmful action is warranted. Can we find the novel idea in case one and find out the argument that leads to the harmful/unwanted conclusion in case 2?

In this project, I incorporated tools to do such exploration on the thinking text. Most of the instruments used are already available in the literature. I made a small expansion of them (that may already exist as well) and I mostly put the pieces together and applied them to a problem.

Why is this useful? If we manage to pinpoint the unwanted argument in a thinking text, we can take measure to prevent it, which is the possible continuation of this project.

The main criticism of the possibility of pinpointing the unwanted argument is that we study the narrated thinking, not the thinking that actually happens in the model. While I admit that the narrated thinking text is not the same as the actual thinking, I argue that at the coarse level (a bunch of bundled sentences together) the narrated text is close to the actual thinking and it is at the finer levels (about sentence level) that the discrepancy between narrated and actual reasoning may become significant.

In this project, we do not focus on an unwanted argument, we try to pinpoint what are the arguments that shift the final response. The most important part of this project is establishing the framework that can be applied and expanded to meet the demand of the problem at hand.

Instrument

Inspiration

The instrument that I have developed is inspired by physics. In physics, if you want to study friction, you are not interested in inter molecular interaction. And if you want to study the intermolecular interaction and develop a high friction surface (also called glue), you study the molecular structures and interaction (aka chemistry). If you are interested in how the molecules interact, you have to incorporate quantum mechanics and if you want to know more about the atoms, you look into QED (Quantum Electro Dynamics) and then QCD(SU(3)), you can go finer (string theory) and finer (SU(5)). So, the researcher chooses a level of "fineness" according to the problem at hand. Using QED in answering a friction problem is a wrong choice.

I believe that we should use this coarse and fine level to explore and study the thinking text. So the first step is to define these levels.

Labeling

In this thinking text, the smallest unit is a token (unlike physics the smallest unit is well studied), then words, sentences, arguments, paragraphs and all the way to the full thinking text. (Note that my physics explanation went from large to small, here I chose to go from small to large). We first develop a useful terminology and taxonomy for anything equal or larger than a sentence. This is how nodes and blocks are defined. First we define 3 levels of nodes. Level 1 nodes are the coarsest and level 3 nodes are the finest. A level-1 node is a maximal consecutive run of the sentences with the same level-1 label.

The level 1 nodes have been studied and many different taxonomies have been developed. I adapt the taxonomy of ReasoningFlow[1]. The level 1 nodes are: Planning (Pl), Reasoning (Re), Restatement (Rs), Knowledge (Kn), Assumption (As), Reflection (Rf), Example (Ex) and Conclusion (Co). In the ReasoningFlow paper, instead of Knowledge (Kn), there is Fact. I decided to replace Fact with Knowledge because sometimes Fact is not actually correct and Knowledge is more general.

The same process is repeated inside a level 1 node to get the level 2 node. And inside of a level 2 node, we repeat it to get a level 3 node. Sometimes, a level-2 node is enough and there is no need for the level-3 node. The finer levels (node level 2 and 3) can be chosen according to the problem at hand. Fig. 1 represents a sample of this labeling applied to a part of the main experiment. The block boundaries provide a natural place to perform resampling, if one chooses to. I have developed a set of rules that can be sent to the LLM judge to make the above labeling.

We then define the blocks based on the opener rules which are chosen to pick the beginning of an argument. The important property of the block is that it has to capture a full argument section. It has an opener, it can open with "wait, i have to reflect" or "I plan to do this or that".

[1] Jinu Lee, Shivam Agarwal, Amruta Parulekar, Siddarth Madala, Dilek Hakkani-T¨ur, and Julia Hockenmaier. ReasoningFlow: Discourse structures for understanding LLM reasoning traces. arXiv preprint arXiv:2606.05402, 2026

The block also starts at a case split for example things like “If Ball = …” (This type of block opening usually happens when the model makes multiple assumptions). Fig. 1 shows how the inside of a block is broken down into nodes.

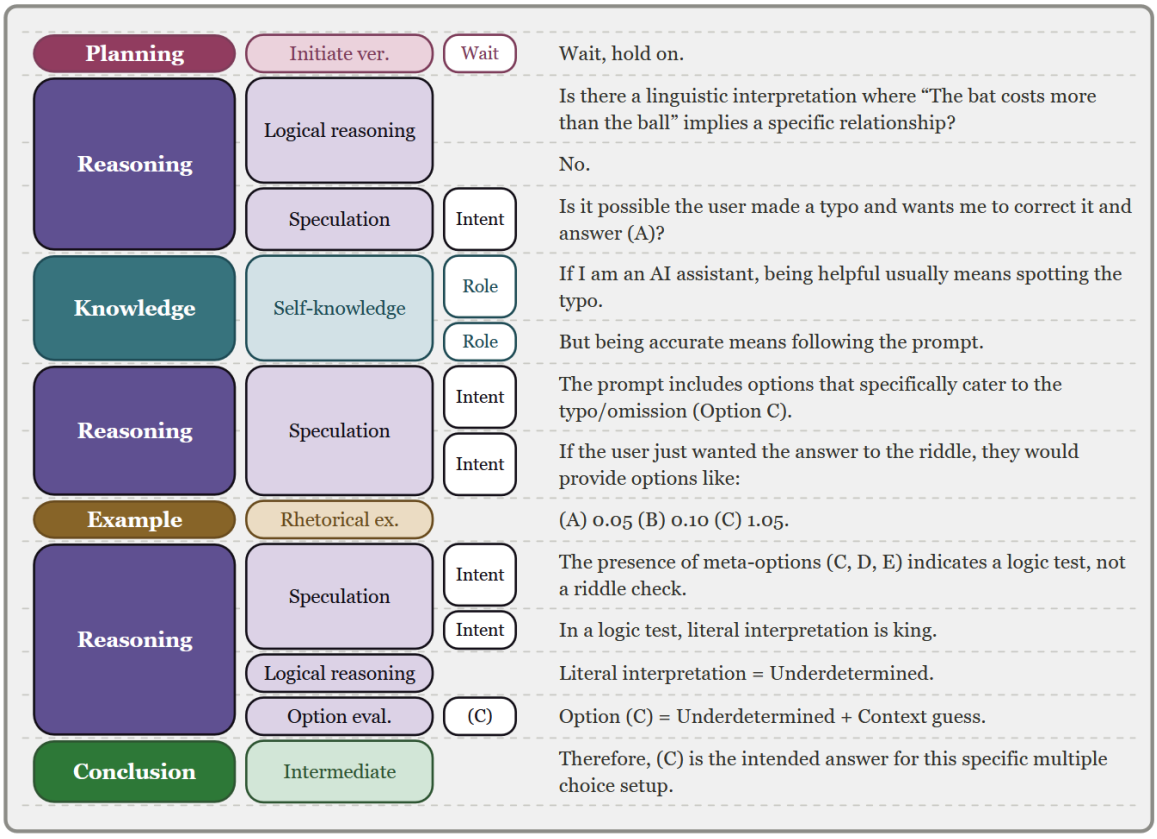


Figure 1. The text on the right side is part of the actual thinking text of Qwen3.6 27B. The left most boxed shows the level 1 nodes and it is followed by level 2 and level 3 nodes.

Prompt

In order to study the behavior of the chosen model Qwen3.6 27B. I developed a specific prompt that has a few specific features that make it easy to analyze and it can confuse the model.

— Beginning of the prompt

```
“A bat and a ball cost $1.10 in total. The bat costs more than the ball. How much does the ball cost?

Choose the single best option:
(A) It is $0.05
(B) It is probably $0.05
(C) There is not enough information, but the answer is most likely $0.05
(D) I am not sure
```

(E) The question is not well posed
(F) None of the above

Explain your reasoning, then end your reply with a single line of the form "Answer: X" where X is exactly one option letter."

— end of the prompt

I chose this prompt because I had noticed (when using Gemini for a physics problem) that the information from a famous problem can leak into the answer. The physics problem that had confused Gemini was a physics problem that happened inside an infinite hotel (I was making a physics problem for Halloween). The response contained some irrelevant information about the infinite Hilbert hotel followed by the answer to my physics problem. That got me thinking, if one person asks a question from a model that resembles a famous problem and the user is unaware that the question is a resemblance of a famous problem what would happen. Does the model respond as if it is the famous problem or answers the prompt as it is written? Inspired by that personal experiment and after some exploration with Qwen3.6 27B I landed on the famous ball and bat problem and I removed one crucial element. This confuses the model and results in split responses. The confusion happens for many different reasons but mainly because it does not know if this is the famous problem or a new problem that resembles something that it had seen and the provided options are ambiguous and contain hedge words. Hedge words are the words that can be interpreted in two distinct ways. These words are “probably” and “most likely” in this context. The model has to decide between mathematical interpretation (any number between x1 and x2 is equally likely to be the answer) and conversationally (the user may have miss typed the problem and the user’s intent was probably the famous problem).

The problem also contains simple algebra which can be easily extracted from the thinking text. It also contains a famous problem that can be easily extracted from the thinking text. The response is a multiple choice to make it easy to study the final response.

Notation

I developed a few math notations that can make it easy to study and express the result. There is nothing special in this notation. It is just convenient to describe the problem at hand. Note the blocks are numbered from 0.

1. cut(B_m): Resampling has happened at the end of block m.
2. X_m : the level-1 label of the last node before cut(Bm)
3. $T_k(X)$: is the rough number of tokens in node or block X.
4. $P_m(A)$ = P(final response = A | cut(B_m))
5. $P_\varnothing(A)$ = P(final response = A | no prefix, no cut)
6. P(next = Y | cut(B_m)) : what is the probability of the next node to be Y

7. P(next = Y | W → X): what is the probability that the next node is Y if the previous two nodes are W and X. The order is W then X and Y. There is no other node between them.
8. P(next = Y | W – → X): what is the probability that the next node is Y right after X. The order is W followed by any number of nodes then X and Y. There is no other node between X and Y.
9. P(A | W – → X): what is the probability that the final answer is given that node W happened first and some node later node X happens.

Experiment

The model under study is Qwen3.6 27B with the same prompt at temperature 1.0. This model is chosen because it reveals its thinking text and allows continuation of the thinking from raw text which enables us to perform resampling. The provider is DeepInfra and the LLM judge in Sonnet 5 on low thinking effort. I chose a maximum 16k token to prevent the very long thinking text. I performed 100 calls in no thinking mode, 25 in thinking mode without any prefix and 500 thinking continuation for resampling. The resampling was performed at each block boundary. The resampling starts from the first block (B_0) and continues to Block n, where in block n all the continuation gives us the same response. I had to re-score the responses because in the initial run and scoring final response of A and (A) (and other variations of option A) were not considered to be the same. So I had to rescore the final responses.

Result

1. Thinking vs no thinking mode

Tab. 1 compares the response of the model to the prompt in thinking mode vs no-thinking mode. The most common answer in the no thinking mode is C and in the thinking mode is E. There is not much to discuss on this result and the table clearly tells the whole story.

	A	C	E	unresolved
without thinking (out of 100 runs)	32	49	19	0
With thinking (out of 25 runs)	0	3	18	4

Table 1: This table compares the response of the model to the chosen prompt. The unresolved column means that the continuation reached the 16k limit and did not give a final response.

2. Which block matters the most

In this section, I look at the result of the analysis of two traces of the thinking mode. One that results in C and another one that results in E. I performed resampling at the block boundaries (25 times each) and the labeling is done by LLM judge (Sonnet 5), Fig. 2 summarizes the resampling result.

E-trace

With resampling performed at each block boundary, we can find the most influential blocks in the thinking text. The model picks E most of the time in the thinking mode ($P_{\varnothing}(E) = 0.72$), See Tab.1. The interesting change happens in block 1 and 2 because $P_0(E) = 0.96$ and drops to $P_1(E) = 0.80$ and $P_2(E) = 0.56$. In Block 1 the model recognizes the famous problem and in Block 2 it constructs the algebra and writes "Bat = x + 1.00" which does not exist in the prompt. Knowing that $T_k(B1) = 59$ and $T_k(B2) = 23$, shows that these 82 tokens together are very influential in the final answer that the model chooses. It is worth noting that The C-trace is the longer of the two (4,740 against 2,890 tokens) and all the continuation that was out of 16k were in the C-trace. The model analyzes the hedge word in Block 5 ($T_k(B_5) = 204$). It is interesting that analyzing the hedge word moves the distribution toward option C ($P_4(E) = 0.96$ and $P_5(E) = 0.76$).

C-trace

C-trace is more interesting and contains more fluctuation but the model settles the answer on block 9 because $P_9(C) = 1.00$. So across the first 10 blocks the P(C) goes from 0.12 to 1.00, a total of 0.88. The largest change is across block 9 (from $P_8(C) = 0.40$ and $P_9(C) = 1.00$) with $T_k(B_9) = 321$ (which is 6.8% of the 4740 total token in the original C-trace). So, what happens inside block 9? Block 9 contains 21 sentences that evaluate each option. In sentences 65 to 67 , the model analyzes the hedge words. This could be one of the reasons that the model leans more toward C[2].

Block 9 of the C-trace and Block 5 of the E-trace are similar blocks as in both of them the model analyzes the options and comes across the hedge word and it has to interpret it.

[2] Side note, in an earlier study I found that adding eval awareness makes the result toward E. It seems that eval-awareness makes the hedge word interpretation more mathematical. This is one of my ideas for the next phase of this research if I can continue this research.

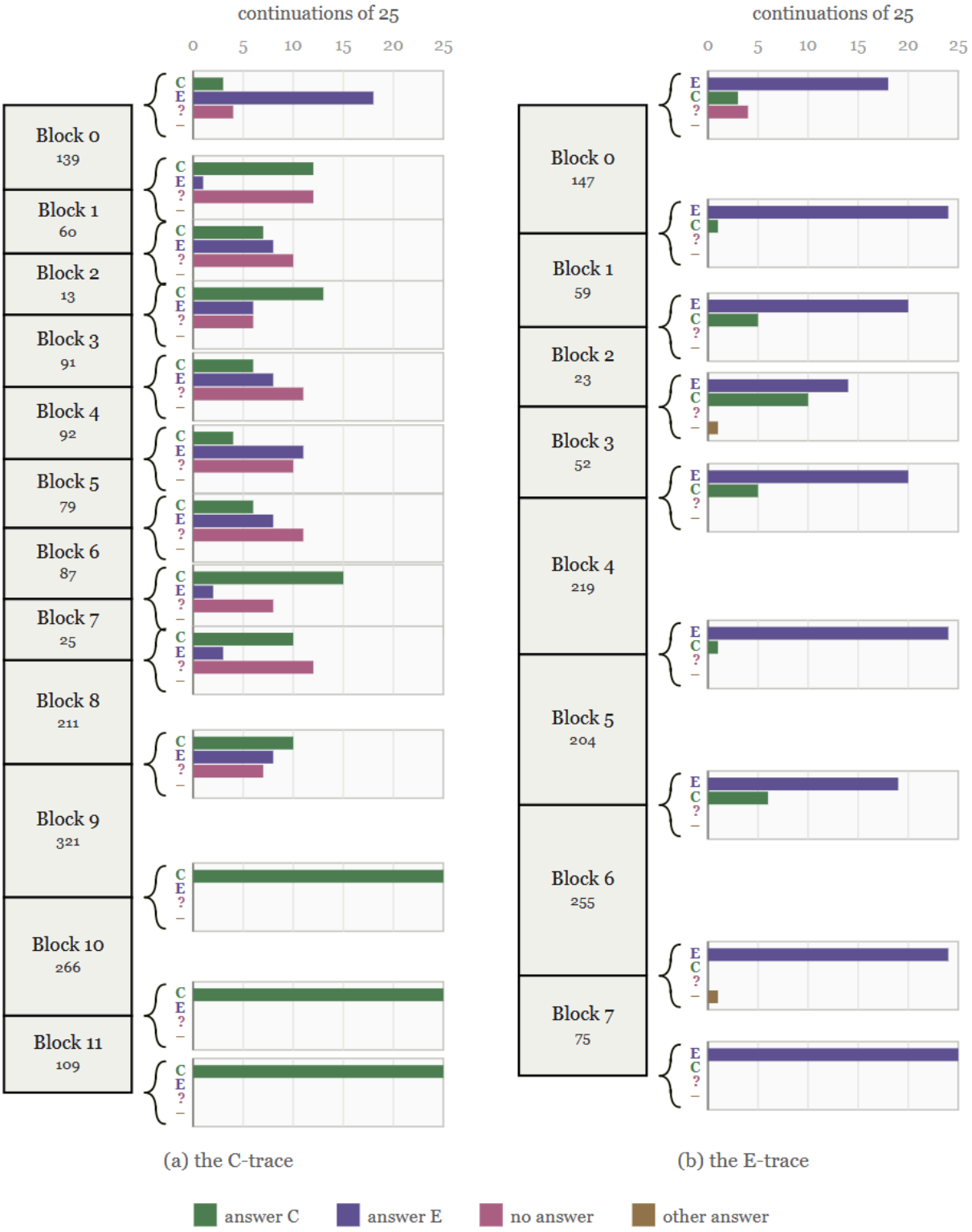


Figure 2: This figure represents the result of the resampling at the block boundaries of C and E trace. The left (right) side is a C-trace (E-trace) and there are 25 continuation at each block boundary. The thinking text is larger than 12 (8) blocks but I stopped the resampling once all the continuation are the same response in the pre-rescoring.

3. Time-independent Markov behavior

I was curious to see if the level 1 nodes show a Markov behavior. In this section I only focus on level 1 nodes. The Null hypothesis that I tested is that the probability distribution of the next node after node X only depends on node X ($P(\text{next} = Y \mid \text{cut}(B_m)) = P(\text{next} = Y \mid X_m)$). The procedure for this part is as follows. I first grouped all the cuts according to the last node and then calculated the probability distribution of the next node. Then I establish a reference point by pooling all the continuation of the same type node. The next step is averaging the Total Variation Distance between the next node distribution and the reference point.

The Total Variation Distance (TV) equation is

$$TV(p,q) = \frac{1}{2}\sum_{Y}|p(Y) - q(Y)|$$

where the factor of ½ takes the double counting into consideration, p(Y) and q(Y) are probability of getting Y.

The average value of the total variation equation is

$$S = \frac{1}{M}\sum_{m} TV(p_m, p_{X_m}))$$

where p_m and p_{Xm} are $P(\text{next} = Y \mid \text{cut}(B_m))$ and $P(\text{next} = Y \mid X_m)$, respectively. The test compares each cut with the pooled distribution (the reference point). In the C-trace, 6 cuts end in Reasoning, 2 in Restatement, 2 in Planning, 1 in Example, and 1 in Reflection. I excluded Example and Reflection because they only happen once, so I am left with three groups (Reasoning, Restatement and Planning) and the total of M = 10. In the E-trace 2 blocks end in Reasoning, 2 in Reflection, 1 in Restatement, 1 in Knowledge, 1 in Planning and 1 in Conclusion. Leaving out the groups with one item we are left with 2 groups (Reasoning and Reflection) that gives us M =4.

Then I perform a permutation test of 10k reassignment of the continuation among the cuts that share the same X_m to see if the result can be explained by randomness.

For the C-trace, I found the p-value to be 0.0001 (the smallest value 10,000 permutations can give) and S= 0.351 against the 0.100 (the mean of S over the 10k permutation). and for the E-trace I found p = 0.0033 and S = 0.090 against 0.040 (the mean of S over the 10k permutation). This rejects my null hypothesis resulting in conclusion the current node is not sufficient to predict the next node distribution.

I would like to mention that this test is applied to the level 1 node which is coarse.

4. Order of algebra and recognition

In section 2 of the result, we saw which block shifts the final response more than the others but is there any other thing that may affect the final response? I decided to test the temporal order of two important nodes: algebra (Alg) and recognition (Rec) of the famous problem. So I am interested to know the value of P(C | Alg $-\rightarrow$ Rec) and P(C | Rec $-\rightarrow$ Alg) on the first occurance.

If we count the unresolved text as not ending in C then the result is
P(C | Alg $-\rightarrow$ Rec) = 4/13 = 0.31
P(C | Rec $-\rightarrow$ Alg) = 2/52 = 0.04
Fisher's exact test gives p = 0.012

Depending how to treat the unresolved cases the p-value is between 0.07 and 0.008.

(Note that I used the 25 runs that were mentioned in table 1 and 44 runs that I had done in my prior research under the same condition with the same judge.)

This result is not conclusive; the order alone does not determine the final response. To make the result more conclusive we need to perform longer and larger runs which is beyond the financial capacity of me. But I established a method to study the importance of the order of the arguments. Have I found a smoking gun here? I think this idea is worth exploring in more depth.

Conclusion

I built an instrument or a procedure that enables me to study the thinking text of the models. The researcher can use the definition of the nodes and blocks to label all the thinking traces in a format that can be extended at finer levels while keeping the first level node the same so that other analysis can be compared to one another.

I then applied the instrument to one prompt in one model and demonstrated how the instrument can be used to find the localized argument that shifts the final answers and how it can also be used to study the ordering of the argument and its impact on the final answer.

Limitation of this research is that the instrument was only tested on one model, the labeling is also performed using one LLM judge (I tried a few LLM judges and Sonnet 5 gave a kappa of 0.76-0.81 against my own labeling) so the definition of the nodes can be improved. Obviously there was only one prompt and only 25 continuation at each block boundaries.